

Laboratorio Calificado 07

2026-1

ÍNDICE

1	Instrucciones.....	3
2	Contexto.....	3
3	Objetivos.....	3
4	Requisitos.....	3
5	Entregables.....	4
6	Rúbrica de Evaluación.....	5

Laboratorio Calificado 07

Programación de la capa de persistencia para el mantenimiento de la entidad 'Unidad Robótica' del Sistema de Gestión de Ensamblaje en una Línea de Producción Automatizada de Mecatrónica

1 Instrucciones

Se ha creado la tarea “Laboratorio Calificado 7” en la sección Laboratorio Calificado 7 de la página del curso-horario en PAIDEIA. Cada alumno deberá entregar un archivo comprimido (formato ZIP) con todos los archivos del desarrollo realizado en el laboratorio. El nombre del archivo comprimido deberá tener el formato:

L7_<Código del alumno de 8 dígitos>.<Extensión del archivo>

Como ejemplo, el nombre de archivo del alumno 20236969 sería “L7_20236969.zip”. Es COMPLETA RESPONSABILIDAD DE CADA ALUMNO el colocar nombres correctos a sus archivos y evitar así confusiones al momento de la calificación.

2 Contexto

En los laboratorios anteriores se desarrollaron:

En los laboratorios anteriores se desarrollaron satisfactoriamente las siguientes capas para la gestión de la flota:

- Capa del modelo: Representada por la clase UnidadRobotica, con atributos como Serial ID, alias, tipo de robot, estado operativo y ubicación física.
- Capa de control: Que gestiona la flota mediante un repositorio en memoria y operaciones CRUD.
- Capa de presentación (GUI): Desarrollada en C++/CLI con Windows Forms, que permite la gestión visual de los activos.

Sin embargo, actualmente los datos de las unidades robóticas no se conservan al cerrar la aplicación, ya que no existe una capa de persistencia que almacene la información de forma permanente.

En este laboratorio, se solicita implementar dicha capa de persistencia para que las unidades del sistema puedan almacenarse y recuperarse automáticamente desde un archivo (por ejemplo, en formato texto plano, CSV, XML o binario). El objetivo es lograr que los datos técnicos y el estado de la flota se mantengan disponibles entre sesiones de ejecución del programa, garantizando la integridad de la información en la línea de producción.

3 Objetivos

- Implementar la capa de persistencia para el sistema de gestión de ensamblaje, permitiendo el almacenamiento permanente de las unidades robóticas.
- Integrar la persistencia con la capa de control (RobotController) y la interfaz gráfica (GUI), asegurando que la lógica de negocio se comunique correctamente con el almacenamiento externo.
- Permitir la carga automática de datos de la flota al iniciar el programa y el guardado automático (escritura en archivo) al realizar operaciones CRUD o al cerrar la aplicación, garantizando que el estado de los robots sea persistente.

4 Requisitos

a) Implementación del mecanismo de persistencia (10 puntos)

- Definir una clase o módulo responsable exclusivamente de la lectura y escritura de los datos de las unidades robóticas en un archivo externo.
- Formato de archivo: El estudiante puede elegir entre texto plano, CSV, XML o binario, asegurando que todos los atributos técnicos de la clase UnidadRobotica se guarden correctamente.
- Arquitectura: Se debe seguir estrictamente el principio de separación de responsabilidades, evitando que la lógica de manejo de archivos se mezcle con la lógica de negocio del controlador o la interfaz gráfica.

b) Integración con el Controller y la GUI (6 puntos)

- Carga Automática: Al iniciar la aplicación, el sistema debe leer el archivo y cargar automáticamente las unidades robóticas en el repositorio en memoria del RobotController.
- Sincronización: Cada operación CRUD realizada desde la interfaz gráfica debe desencadenar la actualización inmediata del archivo de persistencia para evitar la pérdida de datos ante cierres inesperados.
- Calidad de Código: Mantener una estructura modular, limpia y debidamente comentada.

c) Ejecución y pruebas (4 puntos)

- Probar el sistema realizando la siguiente secuencia de operaciones:
 - Inserción de 3 unidades con configuraciones distintas (e.g., Stewart, Brazo Redundante y AGV).
 - Modificación del estado operativo de una unidad (e.g., cambiar de "Operativo" a "En Mantenimiento").
 - Eliminación de una de las unidades registradas.
 - Consulta individual por Serial ID y consulta general de la flota.
- Validación de Persistencia: Cerrar y reiniciar la aplicación; los cambios realizados deben permanecer visibles en la GUI.
- Evidencia: Incluir capturas de pantalla de las operaciones en la GUI y del contenido del archivo de persistencia (abierto en un editor de texto) antes y después de las pruebas.

5 Entregables

- Código fuente completo en C++/CLI, bien estructurado y comentado.
- Capturas de pantalla de la ejecución de las operaciones (inserción, modificación, eliminación, consulta individual y general).

6 Rúbrica de Evaluación

Puntaje Máximo: 20 puntos

Criterio	Excelente	Bueno	Regular	Deficiente	No Cumple
Diseño e implementación del mecanismo de persistencia (10 pts)	Implementa un mecanismo completo y robusto de persistencia. La carga y guardado son automáticos y desacoplados del Controller o Service. (9-10 pts)	Funciona correctamente, pero requiere pasos manuales o presenta leves limitaciones. (7-8 pts)	Persistencia básica con errores en carga o guardado. (5-6 pts)	Persistencia parcial o inestable. (3-4 pts)	No implementa persistencia o el código es erróneo. (0-2 pts)
Integración con Controller y GUI (6 pts)	Total integración entre GUI, Controller y persistencia. CRUD actualiza correctamente el archivo. (5-6 pts)	Integración funcional con pequeños errores o redundancias. (3-4 pts)	Integración parcial: algunas operaciones no actualizan el archivo. (2 pts)	Integración mínima, con errores graves o duplicación de lógica. (1 pt)	No hay integración o el sistema no usa los datos persistidos. (0 pts)
Ejecución y pruebas (4 pts)	El sistema compila y ejecuta correctamente todas las operaciones CRUD, mantiene los datos tras reinicio, y presenta capturas claras de todas las pruebas. (4 pts)	Todas las operaciones funcionan excepto una menor; capturas casi completas. (3 pts)	Varias operaciones fallan; capturas incompletas o poco claras. (2 pts)	El programa compila parcialmente, sin evidencias suficientes. (1 pt)	No compila ni se muestran evidencias de ejecución. (0 pts)